PROPUESTA TÉCNICA · CONFIDENCIAL

Paralelización de Evidencia QA

Propuesta de [illegible] para la validación multiplataforma con [illegible]

Proyecto	Fecha	Versión	Tipo	Estado actual
Intrale Platform	Marzo 2026	v1.0	Investigación técnica · ADR	Secuencial (1 emulador local)

1. Resumen Ejecutivo

Actualmente el proceso de evidencia QA es **100% secuencial**: un emulador Android local (AVD virtualAndroid, API 35), un script `qa-android.sh`, y grabación con `screenrecord`. Esto significa que validar 3 issues con 3 flows cada uno tarda ~30 minutos en ejecución serial.

Esta investigación evaluó 6 estrategias para paralelizar el proceso. La **solución recomendada a corto plazo** es **Maestro Shards con múltiples AVDs locales** (esfuerzo bajo, impacto inmediato, sin cambio de infraestructura). A mediano plazo, integrar **GitHub Actions con matrix strategy** (esfuerzo medio, escalabilidad ilimitada).

⚠️ **Problema actual**

Con el plan de QA compliance (16 issues, ~48 flows de Maestro), el tiempo total en modo secuencial es **~8 horas** de grabación continua en un solo emulador. Esto bloquea el ciclo de entrega.

Issues QA pendientes **16**	Flows estimados **~48**	Tiempo secuencial **~8h**
Tiempo con 3 AVDs **~2.5h**	Tiempo con CI/CD **~45min**	

2. Opciones Evaluadas

★ OPCIÓN RECOMENDADA — Maestro Shards con múltiples AVDs locales

ESFUERZO BAJO

RECOMENDADO Usar `maestro test --shards N` (disponible desde Maestro v1.37.0) con múltiples AVDs corriendo simultáneamente en la misma máquina Windows. Maestro distribuye los flows automáticamente entre los AVDs disponibles.

¿Cómo funciona?

Maestro detecta todos los dispositivos/emuladores conectados vía `adb devices` y distribuye los flows en paralelo cuando se usa `--shards N`. Cada shard corre en un emulador diferente.

```
# Arrancar 3 AVDs simultáneamente
$EMULATOR -avd virtualAndroid -no-window -gpu auto -port 5554 &
$EMULATOR -avd virtualAndroid2 -no-window -gpu auto -port 5556 &
$EMULATOR -avd virtualAndroid3 -no-window -gpu auto -port 5558 &

# Ejecutar todos los flows distribuidos en 3 shards
maestro test qa/generated/maestro/ --shards 3 \
--format junit --output qa/recordings/results.xml
```

Requisito de hardware

Cada AVD consume ~2GB RAM + ~15% CPU. Para 3 AVDs: ~6GB RAM + ~45% CPU. La máquina actual (Windows 11 Pro) debería soportarlo si tiene ≥16GB RAM.

✓ VENTAJAS

- Cero cambio de infraestructura
- Maestro soportado nativamente
- 3x más rápido con 3 AVDs
- Funciona hoy mismo (crear 2 AVDs extra)
- Snapshots por AVD = boot rápido

✗ LIMITACIONES

- Limitado por RAM/CPU del host
- Los videos de shards requieren lógica custom
- Máximo ~4 AVDs simultáneos prácticos
- No escala fuera de la máquina local

Pasos de implementación

1 **Crear 2 AVDs adicionales (30 min)**

Clonar virtualAndroid como virtualAndroid2 y virtualAndroid3. Crear snapshots `qa-ready` en cada uno.

2 **Modificar qa-android.sh (2h)**

Agregar soporte para arrancar múltiples AVDs en paralelo y pasar `--shards` a Maestro. Lógica de screenrecord por shard.

3 **Validar con 3 flows (1h)**

Ejecutar el suite de login/recovery/2FA en modo paralelo. Verificar que los videos se generan correctamente.

OPCIÓN 2 — GitHub Actions con matrix strategy (Mediano plazo)

ESFUERZO MEDIO

VIABLE Usar GitHub Actions con `strategy.matrix` para ejecutar múltiples jobs de QA en paralelo en runners. Cada job corre un subset de flows en un emulador Android en CI.

```
# .github/workflows/qa-parallel.yml (fragmento)
strategy:
matrix:
issue: [1112, 1113, 1114, 1115]
jobs:
qa-validate:
runs-on: ubuntu-latest # Con KVM habilitado
steps:
- uses: reactivecircus/android-emulator-runner@v2
with:
api-level: 34 # API 34 max en GitHub Actions
script: bash qa/scripts/qa-validate.sh ${{ "{{" }} matrix.issue {{ "}}" }}
```

⚠️ **Limitación crítica: Android 15 (API 35)**

El AVD actual es API 35. GitHub Actions `ubuntu-latest` soporta hasta API 34 con `android-emulator-runner`. Para API 35 se requiere runner con KVM y configuración manual (más complejo).

✓ VENTAJAS

- Escala sin límite de hardware local
- Ejecución automática en push/PR
- GitHub almacena artifacts (videos, XML)
- N issues en paralelo gratis en repos públicos
- Integración nativa con el repo actual

✗ LIMITACIONES

- API 34 máximo (nuestro AVD es API 35)
- Sin KVM en Linux runners: emulador lento
- Requiere reescribir qa-validate.sh para CI
- Tiempo de setup del runner: ~5-8 min por job
- Minutos de Actions: repo privado = costo

OPCIÓN 3 — Docker con Android Emulator (Limitado)

ESFUERZO ALTO

PARCIALMENTE VIABLE Usar budtmo/docker-android para correr emuladores Android en contenedores Docker. Soporta hasta **API 34** en versión open source. API 35 requiere licencia Pro.

🚫 **Bloqueante para el proyecto actual**

El AVD de producción es Android 15 (API 35). budtmo/docker-android open source soporta hasta API 34.

No es compatible directamente

. Se podría bajar a API 34 para las pruebas, pero existe riesgo de divergencia de comportamiento UI.

Requisitos de hardware

Requiere **KVM habilitado en el host**. En Windows, esto significa ejecutar Docker dentro de WSL2 con KVM anidado, lo cual no siempre está disponible en máquinas virtuales o cloud.

✓ VENTAJAS (SI SE RESUELVE API 35)

- Aislamiento completo por container
- Reproducibilidad garantizada
- Fácil de escalar horizontalmente
- Funciona bien en CI/CD Linux

✗ LIMITACIONES ACTUALES

- API 35 no soportado en open source
- Requiere KVM (no siempre disponible)
- En Windows: Docker + WSL2 + KVM anidado
- Overhead de configuración alto
- screenrecord dentro de container: complejo

OPCIÓN 4 — iOS Simulator en Docker

NO VIABLE No existe ninguna forma legalmente soportada de correr el simulador de iOS en Docker o en hardware que no sea macOS con Xcode instalado. Apple no ofrece imágenes Docker y el simulador requiere el runtime de Xcode.

Alternativa para iOS

Para CI/CD de iOS se usa macOS runners de GitHub Actions (`macos-14` con Apple Silicon). Costo: 10x el precio de runners Linux. Para el proyecto Intrale, esto está fuera de scope actual ya que las pruebas Maestro se enfocan en Android.

OPCIÓN 5 — Cloud Testing Services (Firebase Test Lab / BrowserStack)

VIABLE A LARGO PLAZO Servicios como Firebase Test Lab, BrowserStack App Automate, o Sauce Labs permiten ejecutar tests Maestro (o Espresso/XCUITest) contra dispositivos reales en la nube, con video recording incluido.

Servicio	Maestro	Video	API 35	Costo estimado
Firebase Test Lab	No nativo (usa Robo/Espresso)	Sí	Sí	~$1-5/test
BrowserStack App Automate	Sí (soporte experimental)	Sí	Sí	~$400/mes (unlimited)
Sauce Labs	No directo (convertir a WebDriver)	Sí	Sí	~$500/mes
Maestro Cloud	**Sí (nativo)**	**Sí**	**Sí**	**~$200/mes (100 runs)**

💡 **Maestro Cloud — la opción más directa**

Maestro Cloud ejecuta flows .yaml nativamente, con video recording automático y API para integración CI/CD. Es la plataforma SaaS oficial de Maestro. Ofrece tier gratuito limitado.

OPCIÓN 6 — Web/Wasm con Playwright en Docker

VIABLE Para la versión Web (Wasm) de la app, Playwright corre perfectamente en Docker headless con `mcr.microsoft.com/playwright`. Esto ya está parcialmente implementado en el proyecto (QATestBase usa Playwright para tests API).

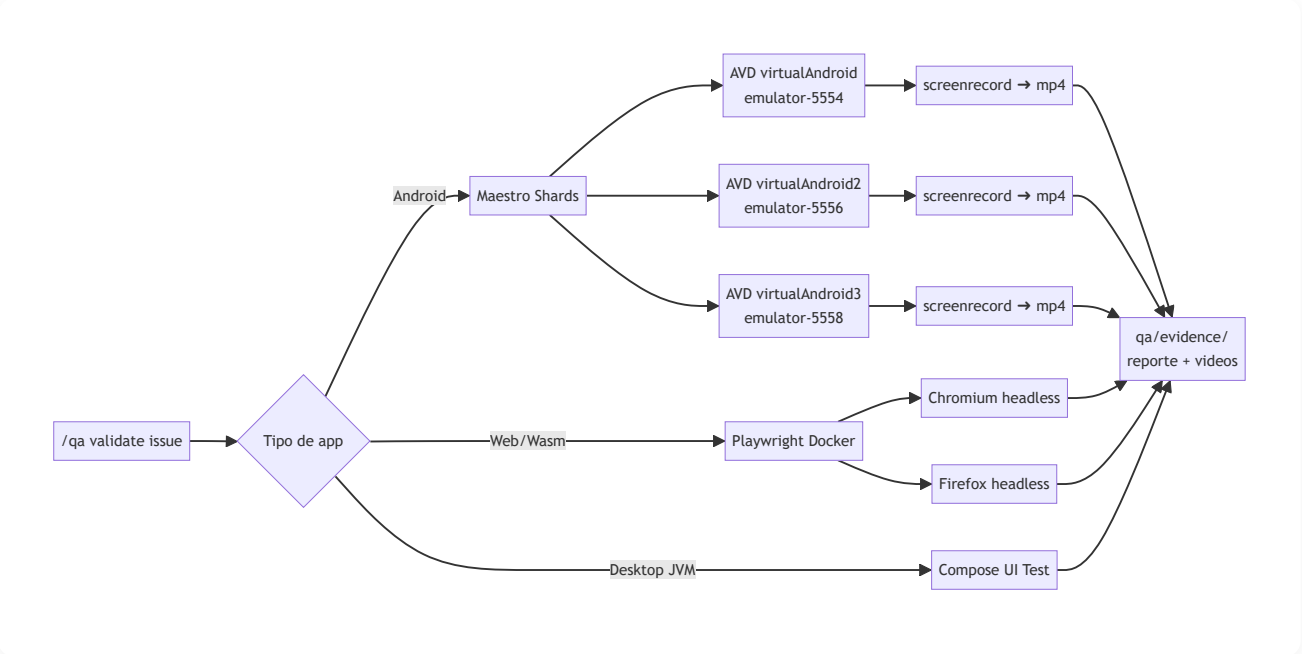
```
# Playwright en Docker — ya funciona para tests API
docker run --rm -v $(pwd):/app mcr.microsoft.com/playwright:v1.41.0-jammy \
npx playwright test --project=chromium
```

Para la versión Wasm, se necesitaría primero implementar el servidor de desarrollo Wasm y luego apuntar Playwright al `localhost` del container.

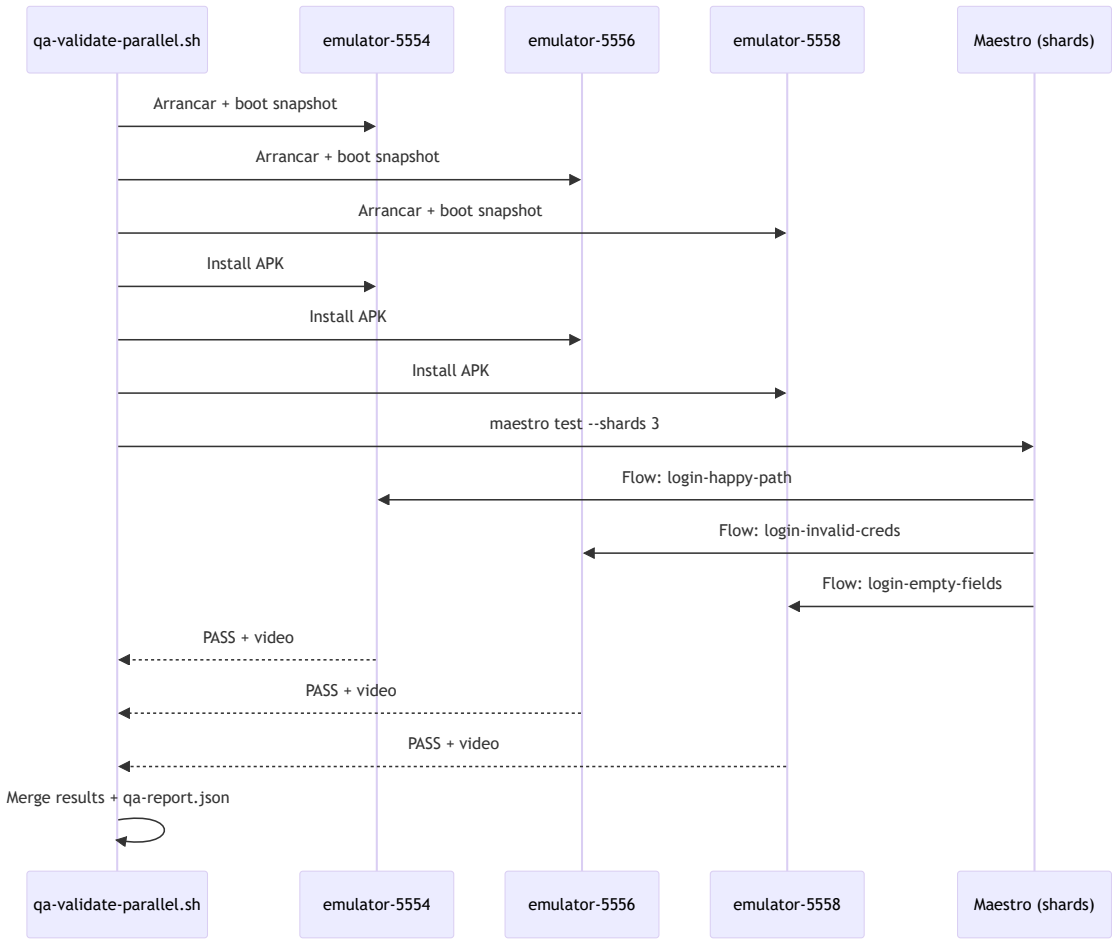
3. Matriz de Decisión

Criterio	Peso	AVDs locales + Shards	GitHub Actions matrix	Docker Android	Maestro Cloud
Compatibilidad API 35	25%	**5 — Nativo**	3 — API 34 max	2 — Requiere Pro	**5 — Soportado**
Esfuerzo de implementación	20%	**5 — Bajo**	3 — Medio	1 — Alto	4 — Bajo-medio
Costo operativo	20%	**5 — $0**	4 — Gratis en público	**5 — $0**	2 — $200/mes
Video recording	20%	4 — Custom script	3 — Artifacts	3 — Complejo	**5 — Nativo**
Escalabilidad	15%	3 — 4 AVDs máx	**5 — Ilimitado**	4 — Alta	**5 — Ilimitado**
SCORE PONDERADO	**100%**	**4.55**	3.65	**2.50**	4.10

4. Arquitectura Propuesta



4.1 Flujo de paralelización con AVDs



5. Plan de Implementación

1 Fase 1: Múltiples AVDs locales (Sprint actual — 4h)

Crear `virtualAndroid2` y `virtualAndroid3` con los mismos specs que `virtualAndroid`. Crear snapshots `qa-ready`. Modificar `qa/scripts/qa-android.sh` para arrancar múltiples AVDs y usar `--shards N`. Validar con los 3 issues actuales (#1112, #1113, #1114).

2 Fase 2: Screenrecord por shard (Sprint +1 — 3h)

Maestro `--shards` no captura screenrecord automáticamente. Implementar wrapper que para cada shard inicia `adb -s emulatorX shell screenrecord` en background y lo mata al terminar. Asociar videos al flow correspondiente.

3 Fase 3: GitHub Actions (Sprint +2 — 1 día)

Crear workflow `.github/workflows/qa-parallel.yml` con matrix strategy por issue. Cada job usa `reactivecircus/android-emulator-runner@v2` con API 34. Storear videos como GitHub Actions artifacts. Trigger: push a branches `agent/*`.

6. Costo-Beneficio

Costo de implementación estimado

Fase 1: AVDs + qa-android.sh	~4h de desarrollo
Fase 2: Screenrecord por shard	~3h de desarrollo
Fase 3: GitHub Actions	~8h de desarrollo
Hardware adicional (RAM)	$0 (ya disponible)
TOTAL	**~15h de desarrollo**

Beneficio esperado

Tiempo actual (16 issues, secuencial)	~8h por sprint QA
Con Fase 1 (3 AVDs)	~2.5h por sprint QA
Con Fase 3 (CI/CD parallel)	~45min por sprint QA
Ahorro acumulado (10 sprints)	**~75h de tiempo de QA**

7. Recomendación Final

Plan de acción recomendado

Implementar en 3 fases progresivas, comenzando por el impacto más inmediato con el menor esfuerzo:

1. **Esta semana: Crear 2 AVDs adicionales + activar Maestro --shards**
   Crear `virtualAndroid2` y `virtualAndroid3`. Modificar `qa-android.sh` para arrancar los 3 y pasar `--shards 3`. Resultado: 3x más rápido sin costo ni nueva infraestructura.

2. **Próxima semana: Screenrecord paralelo**
   Wrapper de screenrecord por emulador/shard. Cada flow tiene su video identificado correctamente en `qa/evidence/{issue}/`.

3. **A mediano plazo: GitHub Actions matrix strategy**
   Escalar a CI/CD con runners Linux + Android emulator. Esto elimina completamente la dependencia del hardware local para la validación QA.

Nota sobre iOS y Docker Android

iOS en Docker: técnicamente imposible (requiere macOS + Xcode, Apple no ofrece imágenes Docker). Docker Android: viable pero bloqueado por API 35 en versión open source. No se recomienda invertir tiempo en estas opciones en el corto plazo.

8. Fuentes y Referencias

- Maestro docs — `maestro test --shards`: maestro.mobile.dev
- budtmo/docker-android: github.com/budtmo/docker-android (API 34 max open source)
- reactivecircus/android-emulator-runner: github.com/ReactiveCircus/android-emulator-runner
- Maestro Cloud: cloud.mobile.dev
- Microsoft Playwright Docker: playwright.dev/docs/docker